

Demo Script

Playwright Beyond the Happy Path

The stage script for the talk. The demos run as **6 continuous sessions**. Each session keeps its terminal and its browser open, and each numbered step is one slide. The number in brackets is the slide number.

All sessions together need about **53 minutes**. Do the preparation before the talk starts. All paths are relative to `src/deploy-or-die-frontend`, if no other path is given.

Preparation — 10 min, before the talk

1. Start the containers one time, so that the image build is not on stage: `pnpm run test:integration`. Keep the report — you need it in session B.
2. Make sure that `ACCESSIBLE_MODE` is `false` in `src/a11yMode.ts`. The accessibility demo needs this.
3. Start the dev server: `pnpm run dev`. It must stay up for all sessions.
4. Install the browser for the C# tests one time: `powershell bin/Debug/net10.0/playwright.ps1 install chromium` in `src/deploy-or-die-dotnet-tests/DeployOrDie.E2E.Tests`.
5. Open three terminals: dev server, test runner, and one spare.

Watch out: the container build needs about 4 minutes. Never do it on stage.

Session 0 · Frame — Slides 2–3 · 2 min

1. **[2]** `npx playwright --version` — show 1.63.0. Say that the .NET bindings are at 1.62.0.
2. **[3]** Show the app: the name field, the deploy button and the result message.
3. Deploy one time and let the audience see the 30 second cooldown.

Watch out: the audience must see the cooldown here. Slide 13 refers back to this moment.

Session A · Fundamentals — Slides 4–7 · 10 min

1. **[4]** `pnpm run test:mocked locators` — green. Then delete one `await` and show the failure message.
2. **[5]** `pnpm run test:ui` — watch mode, pick locator with the mouse, time travel.
3. **[6]** `npx playwright codegen http://localhost:3000` — record one deployment. Name what you must correct: the hard coded name, and the missing negative case.
4. **[7]** `npx playwright show-trace test-results/*/trace.zip` — step back in time, then change to the ARIA tab. The ARIA tab is new in 1.63.

Watch out: close UI mode and codegen before session B. Each of them keeps a browser and a port.

Session B · Mock or real backend — Slides 8–12 · 13 min

1. **[8]** `pnpm run test:mocked` — 17 tests in about 10 seconds, with no backend. One test is red. Say that this is on purpose, and that session D explains it.
2. **[9]** `pnpm run test:integration` — the containers are warm, then 4 tests run against the real ASP.NET Core API and PostgreSQL.
3. **[10]** Show `{ lock: 'deploy-db' }` in `tests/deploy-integration.spec.ts`. Run `npx playwright test --project=integration --workers=4` and show in the Timeline that the locked tests do not overlap.
4. **[11]** `pnpm run test:report` — open **Speedboard**, then **Timeline**. Most of the time is the container start, not the tests.
5. **[12]** `pnpm run test:retries` — this turns the retries on and the red accessibility test retries twice. Point at the last two lines: both retries run *after* the whole suite, one after the other. Then comment `retryStrategy` out, run again, and show that the first retry now runs in the middle of the suite. Put the line back.

Watch out: do not remove the lock on stage to force a failure. These four tests write no shared row, thus they do not reliably conflict. Show the Timeline instead.

Session C · Time control — Slide 13 · 3 min

1. **[13]** Point back to the cooldown from session 0. A test must not wait 30 seconds.
2. `pnpm run test:clock` — 3 tests in about 2 seconds.
3. Walk through the three calls: `install()`, then `runFor(5_000)`, then `fastForward('00:30')`.

Watch out: `install()` must be the first clock call, and it must come before `page.goto()`.

Session D · Accessibility — Slides 14–15 · 7 min

1. **[14]** `pnpm run test:a11y` — red. Three rules fail: `button-name`, `color-contrast` and `label`.
2. Open the report and show the attached `axe-results.json`.
3. Set `ACCESSIBLE_MODE = true` in `src/a11yMode.ts`. Run again — green.
4. **[15]** Change the `h1` to an `h2` in `src/App.tsx`. The axe test stays green, but the ARIA snapshot fails. Say why the two checks find different faults.
5. Undo both changes.

Watch out: set `ACCESSIBLE_MODE` back to `false` after the talk, or the demo is gone the next time.

Session E · .NET — Slides 16–18 · 9 min

1. **[16]** The dev server from the preparation must still run on port 3000. `dotnet test` — 5 tests, 1 red. Put `DeployTests.cs` and `tests/cooldown.spec.ts` side by side. Same locators, same `auto-wait`, same `Clock API`. Only the names use the .NET style.
2. **[17]** Walk the two boxes. Say the one sentence that matters: the browser API is the same, the test runner is not. Point back to slide 10 and slide 12 — test locks and isolated retries are Playwright Test only. Then run `npx playwright codegen --target csharp-xunit http://localhost:3000` and record one click — the recorder writes C#.
3. **[18]** Show the red `AccessibilityTests.cs`. Both languages report the same three rules: `button-name`, `color-contrast` and `label`. Put the C# output next to the TypeScript output from session D.

Watch out: `global.json` selects the Microsoft Testing Platform runner. Without it, `dotnet test` fails on the .NET 10 SDK. The C# tests need their own browser: `pwsh bin/Debug/net10.0/playwright.ps1 install chromium`.

Session F · AI and agents — Slides 19–21 · 9 min

1. **[19]** `npx playwright mcp` — the server is part of Playwright since 1.62.
2. Give the agent one task: explore `http://localhost:3000`. Let it find the `cooldown` with no hint from you.
3. **[20]** Show `npx playwright init-agents --loop=claude` and name the three agents: `planner`, `generator`, `healer`.
4. Change the button label from `Deploy Now` to `Ship It` in `src/App.tsx`. Let the healer repair the test.
5. **[21]** Show the three defects in the wrong example on the slide. Open `tests/locators.spec.ts` next to it. Close with the rule: the agent explores, the human reviews.

Watch out: this session needs network and a model. Record a fallback video before the talk.

Command reference

SESSION	COMMAND
Preparation	<code>pnpm run test:integration && pnpm run dev</code>
A · Fundamentals	<code>pnpm run test:mocked locators · pnpm run test:ui · npx playwright codegen http://localhost:3000</code>

SESSION	COMMAND
A · Fundamentals	<code>npx playwright show-trace test-results/*/trace.zip</code>
B · Mock or real	<code>pnpm run test:mocked · pnpm run test:integration · pnpm run test:retries · pnpm run test:report</code>
B · Mock or real	<code>npx playwright test --project=integration --workers=4</code>
C · Time control	<code>pnpm run test:clock</code>
D · Accessibility	<code>pnpm run test:a11y</code>
E · .NET	<code>dotnet test (in src/deploy-or-die-dotnet-tests/DeployOrDie.E2E.Tests) — 5 tests, 1 red</code>
F · AI and agents	<code>npx playwright mcp · npx playwright init-agents --loop=claude</code>

